

Inhaltsverzeichnis

1. Arrays	1
2. Collections	1
2.1. Java Collections API	2
2.2. Spezielle Java-Collection-Klassen	3
2.3. Synchronized Wrappers	3
2.4. Unmodifiable Wrappers	3

1. Arrays

Laut Java-Dokumentation ist ein Array ein **Objekt, das eine feste Anzahl von Werten desselben Typs** enthält. Die Elemente eines Arrays sind indiziert, was bedeutet, dass mit Zahlen (sogenannten **Indices**) auf sie zugegriffen werden kann.

Ein Array kann als eine nummerierte Liste von Zellen betrachtet werden, wobei jede Zelle eine Variable ist, die einen Wert enthält. In Java beginnt die Nummerierung bei (*Index*) **0**. Ein Array erkennt man an den **eckigen Klammern []**.

```
Index: { 0 , 1 , 2 , ... }
        :   :   :
String[] array = new String[] { "A", "B", "C" , ... };
```

Es gibt Arrays vom primitiven Typ und Arrays von komplexen Objekttypen. Das bedeutet, dass Arrays vom Typ **int**, **float**, **boolean** usw. verwendet werden können, aber auch Arrays vom Typ **String**, **Object** und benutzerdefinierte Typen.

Ein paar **Beispiele**:

```
int[] arr1 = new int[4];           // feste Länge
int[] arr2 = new int[] {1, 2, 3, 4}; // mit initialen Werten
int[] arr3 = {1,2,3,4};           // ohne 'new'

String[] arr4 = new String[3];
String[] arr5 = new String[]{};
String[] arr6 = {"A", "xdre", "Frg%5"};
```

2. Collections

Siehe z.B. → [Java Collections Framework auf Wikipedia](#)

Die wichtigsten Auswahlkriterien:

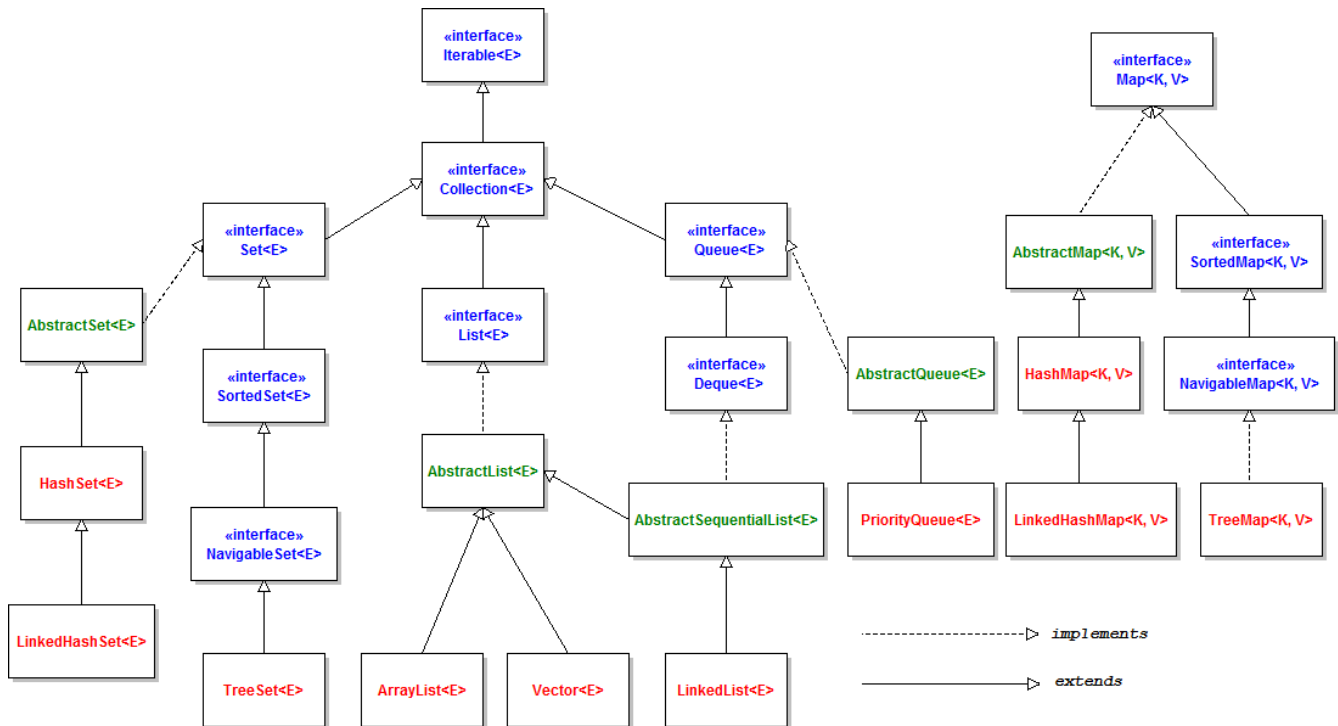
- Position von Elementen (*numerischer index*)

- Möglichkeit zur Ordnung von Elementen (z.B. *insertion order*)
- Möglichkeit von `null` Elementen oder Duplikaten (*Set* vs. *List*)



→ Duplikate in Listen? Wann ist ein Listenelement ein Duplikat? Technisch gleich oder logisch gleich?

- Zugriff auf Elemente anhand eines Schlüssels (*List* vs. *Map*)



2.1. Java Collections API

Die Java-Collection-Interfaces bilden die Grundlage des Java Collections Frameworks. Alle zentralen Collection-Interfaces sind generisch. Zum Beispiel `public interface Collection<E>`. Die Syntax `<E>` steht für Generics (*mehr dazu später im Kapitel Generics*).

1. **Collection-Interface** Dies ist die Wurzel der Collection-Hierarchie. Eine Collection repräsentiert eine Gruppe von Objekten, die als ihre Elemente bezeichnet werden. Einige grundlegende Operationen werden bereitgestellt.
2. **Iterator-Interface** Das Iterator-Interface stellt Methoden bereit, um über die Elemente einer Collection zu iterieren. Iteratoren ermöglichen es dem Aufrufer, während der Iteration Elemente aus der zugrunde liegenden Collection zu entfernen.
3. **Set-Interface** Ein Set ist eine Collection, die keine Duplikate, doppelte Elemente, enthalten darf. Die Java-Plattform enthält drei allgemeine Set-Implementierungen: `HashSet`, `TreeSet` und `LinkedHashSet`.
4. **List-Interface** Eine List ist eine geordnete Collection und kann doppelte Elemente enthalten. Auf jedes Element kann über seinen Index zugegriffen werden. Listen haben eine dynamische Länge. `ArrayList` und `LinkedList` sind Implementierungsklassen des List-Interfaces.
5. **Queue-Interface** Eine Queue ist eine Collection, die mehrere Elemente vor deren Verarbeitung hält. Neben den grundlegenden Collection-Operationen bietet eine Queue zusätzliche Einfüge-,

Entnahme- und Überprüfungsoperationen. Queues ordnen Elemente typischerweise (aber nicht zwingend) nach dem FIFO-Prinzip („first-in-first-out“).

6. **Deque-Interface** Eine lineare Collection, die das Einfügen und Entfernen von Elementen an beiden Enden unterstützt. Der Name **deque** ist die Kurzform von „**double-ended queue**“.
7. **Map-Interface** Eine Java-Map ist ein Objekt, das Schlüssel zu Werten zuordnet. Eine Map darf keine doppelten Schlüssel enthalten; jeder Schlüssel kann höchstens einem Wert zugeordnet sein. Die Java-Plattform enthält drei allgemeine Map-Implementierungen: **HashMap**, **TreeMap** und **LinkedHashMap**.
8. **SortedSet-Interface** Ein SortedSet ist ein Set, das seine Elemente in aufsteigender Reihenfolge hält. SortedSets werden für natürlich geordnete Mengen verwendet.
9. **SortedMap-Interface** Eine Map, die ihre Zuordnungen in aufsteigender Schlüsselreihenfolge verwaltet. Dies ist das Map-Äquivalent zu SortedSet. SortedMaps werden für natürlich-geordnete Sammlungen von Schlüssel/Wert-Paaren verwendet.



Übrigens, das ist eine beliebte Prüfungsfrage: „*Welche Eigenschaften hat das Collections-Interface ...?*“

2.2. Spezielle Java-Collection-Klassen

Das Java-Collections-Framework enthält viele Implementierungsklassen für die Interfaces. Die gängigsten Implementierungen sind:

1. **HashSet**-Klasse
2. **TreeSet**-Klasse
3. **ArrayList**-Klasse
4. **LinkedList**-Klasse
5. **HashMap**-Klasse
6. **TreeMap**-Klasse

2.3. Synchronized Wrappers

Die **synchronisierten** Wrapper fügen einer beliebigen Collection eine automatische Synchronisierung (Thread-Sicherheit) hinzu. Jedes der sechs zentralen Collection-Interfaces – Collection, Set, List, Map, SortedSet und SortedMap – besitzt eine statische Fabrikmethode, die eine synchronisierte (thread-sichere) Collection zurückgibt, die auf der angegebenen Collection basiert.

2.4. Unmodifiable Wrappers

Unmodifiable Wrapper entziehen einer Collection die Möglichkeit, verändert zu werden. Sie fangen alle Operationen ab, die die Collection modifizieren würden, und werfen stattdessen eine **UnsupportedOperationException**.

Die Hauptanwendungsfälle sind:

- Um eine Collection unveränderlich zu machen, sobald sie aufgebaut ist
- Um bestimmten Clients nur Lesezugriff auf Datenstrukturen zu geben (eine Referenz auf die ursprüngliche Collection behalten, aber eine Referenz auf den Wrapper ausgeben)
- Um `ConcurrentModificationException` zu vermeiden

Demos:

Zu finden in:

```
→ src/test/java/de/dhbw/demo/CollectionsDemoTest.java
```

- Demo 1: Eine einfache Liste mit 'primitiven' Werten
- Demo 2: Eine einfache Liste mit 'komplexen' Werten (Objektinstanzen)
- Demo 3: Key-Value-Paare mit Maps
- Demo 4: Un-/Sortierte Map
- Demo 5a: Un-/Modifiable Lists
- Demo 5b: Un-/Modifiable Lists (alternative Implementierung)

Übungen:

Die **Übungen** werden wie immer in Form von **Unit-Tests** in folgendem *Package* implementiert:

```
→ src/test/java/de/dhbw/exercise/CollectionsExerciseTests.java
```

Die **Testobjekte**, also die zu bearbeitenden Klassen, Interfaces oder anderer Sourcecode sollen ggf. getrennt, nämlich hier umgesetzt werden:

```
src/main/java/de/dhbw/exercise
```

Übung 1

Listen: Schreibe einen Test, in dem eine konkrete Implementierung von `List` und den Datentyp `Integer` benutzt wird. Befülle diese mit beliebig vielen Einträgen. Summiere alle Listeneinträge mithilfe einer `for-each` Schleife und teste das Ergebnis.

Übung 2

Maps: Schreibe einen Test, in dem eine konkrete Implementierung von `SortedMap` benutzt wird. Befülle diese mit mindestens 5 Einträgen, nutze dazu `String` sowohl für den Schlüssel (K) als auch für den Wert (V). Überprüfe die Haupt-Charakteristik "*natürliche Sortierung*" dieses Map-Typs.